

DSA

- Typically computers make 10^8 operations/sec
- $\log n \rightarrow$ logarithmic
- $n^2, n^3 \rightarrow$ polynomial
- $2^n \rightarrow$ exponential

Typical functions and n values for which they are optimal:

(logarithmic)	(linear)	(linearithmic)				
$\log n$	n	$n \log n$	n^2	n^3	2^n	$n!$
↓	↓	↓	↓	↓	↓	↓
$n = 2^{10^8}$	$n = 10^8$	$n = 10^7$	$n = 10^4$	$n = 100$	$n = 35$	$n = 10$
			(Quadratic)	(Cubic)	(exponential)	(factorial)

What is n here?

- For sorting and other on arrays $\rightarrow n = \text{array size}$
- Combinatorial problems $\rightarrow n = \text{number of objects}$
- Graphs $\rightarrow n = \text{number of vertices, number of edges}$
- For a number $\rightarrow \text{number of digits}$

$$(\text{int}) (\log_b n) + 1$$

Eg., $\log_{10} n + 1$ in decimal system

$$(\log_{10} 100) + 1 = 2 + 1 = 3 \text{ digits}$$

$$(\log_{10} 1254) + 1 = 3 + 1 = 4 \text{ digits}$$

Can be used for finding number of digits in a number represented in any base system.

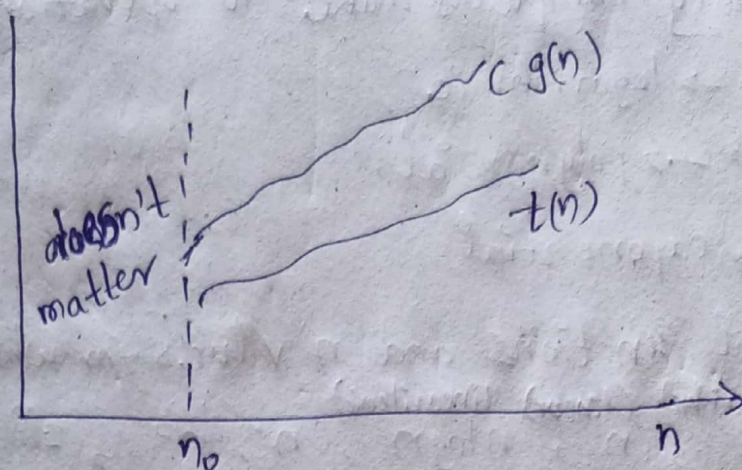
Big O

Time complexity $t(n)$ is said to be $O(g(n))$ if we can find suitable constants C and n_0 .

So that $Cg(n)$ is an upper bound for $t(n)$ for

n beyond n_0

i.e., $O(g(n))$ means $t(n) \leq c g(n)$ for $n \geq n_0$



It is also called asymptotical analysis as we are only concerned with n that is very large.

• If $f_1(n)$ is $O(g_1(n))$

$f_2(n)$ is $O(g_2(n))$

then $f_1(n) + f_2(n)$ is $O(\max(g_1(n), g_2(n)))$

Time Complexity: Time taken by an algorithm as a function of length of input.

Big O $\rightarrow$ worst case complexity.

Eg: $y(n) = 3n^2 + 5n \Rightarrow O(n^2)$

$$y(n) = n + 100 \log n \Rightarrow O(n)$$

$$y(n) = 3n^3 + 4n^5 \Rightarrow O(n^5)$$

$$y(n) = 100000 \Rightarrow O(1) \Rightarrow \text{constant}$$

$$y(n) = \frac{n}{4} \Rightarrow O(n)$$

$$y(n) = \frac{n+100}{4} \text{ (or) } 5n + 3 \log n \Rightarrow O(n)$$

$$y(n) = 5n^2 + 7n + 2 \text{ (or) } 3n^2 + \log n \text{ (or) } \frac{n^2}{100} \Rightarrow O(n^2)$$

* Use of bitwise operations is fast and efficient

• Subtraction in computers is addition of two's complements. So no concept of subtraction.

* $\sim$ operator: inverts all bits

predicting value: If we are doing 2's complement it should be negative of the number but we are doing just 1's complement So 1 less than that (negative number)

in general,

$$\sim n = -n - 1 = \text{one less than the negative of the number}$$

E.g. $\sim 4 = -4 - 1 = -5$

$$\sim 1000 = -1000 - 1 = -1001$$

$$\sim(-4) = 4 - 1 = 3$$

* $\gg$ operator: right shift of bits (divide by 2^k)

E.g. $12 \gg 2 = \frac{12}{2^2} = 3$

$$12 \gg 1 = \frac{12}{2^1} = 6$$

$$10 \gg 2 = \frac{10}{2^2} = 2$$

$$5 \gg 2 = \frac{5}{2^2} = 1$$

* $\ll$ operator: left shift of bits (multiply by 2^k)

E.g. $12 \ll 2 = 12 \times 2^2 = 48$

$$5 \ll 1 = 5 \times 2^1 = 10$$

Trick to check even and odd number (Bit masking)

<pre>if ((n & 1) == 0) even; else odd</pre>		<pre>if ((n & 1) == 1) odd; else even;</pre>
---	--	--

~~It is~~ $n \& 000001$

Here we are checking last bit of n . If it is

1 $\rightarrow$ odd

0 $\rightarrow$ even

Swapping two variables

$$a = a \wedge b$$

$$b = a \wedge b$$

$$a = a \wedge b$$

It is based on the fact that XOR is its own inverse.

$$\text{i.e., } a \wedge 0 = a$$

$$a \wedge a = 0$$

$$\text{So, } a \wedge b \wedge a = (a \wedge a) \wedge b = b$$

$$b \wedge a \wedge b = (b \wedge b) \wedge a = a$$

Number of digits in a number:

$$(\log_b n) + 1 \Rightarrow \text{where } n \rightarrow \text{number}$$

$b \rightarrow$ base system in which the number is represented

In java

for decimal number,

$$(\text{int}) \text{Math.log}_{10}(n) + 1$$

for binary number,

$$(\text{int}) (\log(n) / \log(2)) + 1$$

```
import static java.lang
.Math
```


for any base system, b

$$(\text{int}) (\log(n) / \log(b)) + 1$$

here $\log n = \log_e n$
also $\frac{\log n}{\log b} = \log_b n$
for any base

Swapping Without third variable:

$$\begin{aligned} a &= a \oplus b \\ b &= a \oplus b \\ a &= a \oplus b \end{aligned}$$

$$\begin{aligned} a &= a + b ; a = a + b \\ b &= a - b ; b = (a + b) - b = a \\ a &= a - b ; a = (a + b) - a = b \end{aligned}$$

Bit Masking

Find i th bit (from left starting from zero index)

$$\text{mask} = 1 \ll i \quad (\text{or}) \quad \text{mask} = 2^i$$

$$\text{number} \& \text{mask} \Rightarrow \begin{cases} \text{non-zero} \Rightarrow 1 \\ \text{zero} \Rightarrow 0 \end{cases}$$

Set i th bit (set i th bit to 1)

$$\text{mask} = 1 \ll i$$

$$\text{number} = \text{number} | \text{mask} \quad (\text{i} \text{th bit is set to 1})$$

Clear i th bit (set i th bit to 0)

$$\text{mask} = \sim(1 \ll i)$$

$$\text{number} = \text{number} \& \text{mask}$$

Find the number of bits to change to change a number A to B .

Before: It is to compare last bits of A and B and right shift by each time

while ~~at least~~ ^{any} ~~both~~ of them are non-zero (until both of them become zero)

```
int a, b, Count = 0;
```

```
do {
```

```
    if((a & 1) != (b & 1))
```

```
        Count ++;
```

```
    a = a >> 1; // fill MSB with 0
```

```
    b = b >> 1;
```

```
} while (a > 0 || b > 0)
```

```
(or) for (int i = 0; i < 32; i++) {
```

```
    if((a & 1) != (b & 1))
```

```
        Count ++;
```

```
    a = a >> 1;
```

```
    b = b >> 1;
```

```
}
```

Optimal Solution:

Do, $n = a \oplus b$

Now find the number of set bits in n .

(n) will be the number that has set bits which are different for a and b .

*So the problem is reduced finding number of set bits in a number (binary representation).

method 1: $O(\log n)$

```
int n = number, Count = 0;
```

```
while (n > 0) {
```

```
    if (n % 2 == 1)
```

```
        Count ++;
```

```
    n = n / 2;
```

```
}
```


method 2: $O(\log n)$ (but faster as we are using bit manipulation (*) bitwise op)

```

while (n > 0) {
    if (n & 1 == 1)
        count++;
    n = n >> 1;
}

```

method 3: $O(\text{number of 1's in } n)$

```

count = 0
while (n > 0) {
    count++;
    n = n & (n - 1);
}

```

E.g. $n = 21$
 $= 10101$

start
 $\text{count} = 0$
 as $n > 0$
 $\text{count} = 1$
 $n = n \& (n - 1) = 10101 \& 10100$
 $n = 10100$

as $n > 0$
 $\text{count} = 2$
 $n = n \& (n - 1) = 10100 \& 10011$
 $n = 10000$

as $n > 0$
 $\text{count} = 3$
 $n = n \& (n - 1) = 10000 \& 01111$
 $n = 0000$

$\therefore \text{count} = 3$

As we can see, each time we do $n \& (n - 1)$, it makes the rightmost set bit to 0. thus counting the number of 1's.

Euclid GCD

Recursive

```

long gcd (long a, long b) {
    if (a == 0)
        return b;
    return gcd (b % a, a);
}

```

Time:

Best case: $O(1)$

if b is exactly divisible by a .

worst case: $O(\log n)$

when a, b are consecutive fibonacci.

Average case: $O(\log(\min(a, b)))$

Here Space complexity
Same as time
Complexity. Since
It take space
for method calls.

Iteratively:

~~long gcd(long a,~~

long gcd (long a, long b) {

long temp;

while (a != 0) {

temp = a;

a = b % a;

b = temp;

}

return b;

}

Modular Arithmetic

$$*(a+b) \% m = (a \% m + b \% m) \% m$$

$$*(a-b) \% m = (a \% m - b \% m) \% m$$

$$*(a \cdot b) \% m = (a \% m \cdot b \% m) \% m$$

Generally asked to return mod of 10^9+7

$$= 1000000007$$

3 Steps for Recursion

i) Find the base case

ii) Find the relation between the problem and subproblem.

iii) Generalise the relation

(E.g) printing "Hello" for n-time

```
void PrintHello (int n) {
```

```
    if (n == 0)
        return;
```

```
    print("Hello");
```

```
    PrintHello(n-1);
```

```
}
```

Here, base case : $n == 0$

Generalized relation : $f(n) = f(1) + f(n-1)$

(E.g) Sum of N natural numbers.

```
int Sum (int n) {
```

```
    if (n == 1)
        return 1;
```

```
    return n + Sum(n-1);
}
```

base case : $n == 1$

relation : $f(n) = n + f(n-1)$ where $f \rightarrow \text{sum}$

So, $\text{sum}(4) = 4 + \text{sum}(3)$

here $\text{sum}(3)$ will be computed in the same fashion.
just have faith that it will be computed and
returned, don't think "How?"

"Recursive Leap of Faith!"